Type Soundness for Path Polymorphism

Andrés Ezequiel Viso[1,2]

joint work with Eduardo Bonelli[1,3] and Mauricio Ayala-Rincón[4]

[1]CONICET, Argentina

[2]Departamento de Computación, FCEyN, UBA, Argentina

[3]Departamento de Ciencia y Tecnología, UNQ, Argentina

[4]Departamentos de Matemática e Ciência da Computação, UnB, Brasil

Logic and Foundations of Programming Languages Day
UBA, Buenos Aires, Argentina – May 21, 2018

Index

1. Motivations
   - PPC
   - Typing Error
2. Type System
   - Compatibility
   - Safety
3. Conclusions
4. Future Work

Pure Pattern Calculus

"Every *data structure* is either an *atom* or a *compound* built by application"[Jay09]

Devise a type system for PPC [JK09]

- Pattern Matching

Example (Pattern Matching)

$$
\begin{array}{lll}
map \to_{\{map\}} f \to_{\{f\}} (\ \mathtt{nil} & \to_{\{\}} & \mathtt{nil} \\
\quad\quad\quad\quad\quad\quad\quad\quad \mid \ \mathtt{cons}\,(\mathtt{vl}\,x)\,xs & \to_{\{x,xs\}} & \mathtt{cons}\,(\mathtt{vl}\,(f\,x))\,(map\,f\,xs) \ )
\end{array}
$$

Pure Pattern Calculus

"Every *data structure* is either an *atom* or a *compound* built by application"[Jay09]

Devise a type system for PPC [JK09]

- Pattern Matching
- Path Polymorphism

Example (Generic update query)

$$
\begin{array}{lllll}
upd \to_{\{upd\}} f \to_{\{f\}} & (& \mathtt{vl}\, y & \to_{\{y\}} & \mathtt{vl}\,(f\, y) \\
 & \mid & x\, y & \to_{\{x,y\}} & (upd\, f\, x)\,(upd\, f\, y) \\
 & \mid & w & \to_{\{w\}} & w \qquad\qquad\qquad\qquad) \\
\end{array}
$$

Pure Pattern Calculus

"Every *data structure* is either an *atom* or a *compound* built by application"[Jay09]

Devise a type system for PPC [JK09]

- Pattern Matching
- Path Polymorphism
- Pattern Polymorphism

Example (Generic update query)

$$
\begin{array}{lllll}
upd \to_{\{upd\}} z \to_{\{z\}} f \to_{\{f\}} (& z\,y & \to_{\{y\}} & z\,(f\,y) & \\
 & \mid x\,y & \to_{\{x,y\}} & (upd\,z\,f\,x)\,(upd\,z\,f\,y) & \\
 & \mid w & \to_{\{w\}} & w &)
\end{array}
$$

Pure Pattern Calculus

"Every *data structure* is either an *atom* or a *compound* built by application"[Jay09]

Devise a type system for PPC [JK09]

- Pattern Matching
- Path Polymorphism
- Pattern Polymorphism
- Matching failure is handled explicitly

Example (Generic update query)

$$
\begin{array}{lllll}
upd \to_{\{upd\}} z \to_{\{z\}} f \to_{\{f\}} (& z\,y & \to_{\{y\}} & z\,(f\,y) & \\
 & \mid x\,y & \to_{\{x,y\}} & (upd\,z\,f\,x)\,(upd\,z\,f\,y) & \\
 & \mid w & \to_{\{w\}} & w &)
\end{array}
$$

Calculus of Applicative Patterns

We focus on a static restriction of PPC we call CAP

- Static Patterns
- Built-in case constructor (handles matching failure)
- Path Polymorphism

Example (Generic update query)

$$
\begin{array}{lllll}
upd \to_{\{upd\}} f \to_{\{f\}} & (& \mathtt{vl}\, y & \to_{\{y\}} & \mathtt{vl}\,(f\, y) \\
 & \mid & x\, y & \to_{\{x,y\}} & (upd\, f\, x)\,(upd\, f\, y) \\
 & \mid & w & \to_{\{w\}} & w \qquad\qquad\qquad\qquad) \\
\end{array}
$$

- Evaluation from left to right

$$
\frac{\{\!\{u/p_i\}\!\} = \mathtt{fail} \text{ for all } i < j \quad \{\!\{u/p_j\}\!\} = \sigma_j \quad j \in 1..n}{(p_i \to_{\theta_i} s_i)_{i \in 1..n}\, u \to \sigma_j s_j}\ (\beta)
$$

Typing Error

"Patterns are partial descriptions of data structures" [Jay09]

Matching failure $\neq$ Typing error

$$(\texttt{nil} \rightarrow_{\{\}} \texttt{true})\,(\texttt{cons}\, x\, \texttt{nil})$$

Not all applications $(p \rightarrow_\theta s)\, u$ make sense

p	u	
c	d	✓
c	$d\ t$	✓
c	$q \rightarrow_{\theta'} t$	✗

p	u	
$p_1\ p_2$	d	✓
$p_1\ p_2$	$d\ t$	✓
$p_1\ p_2$	$q \rightarrow_{\theta'} t$	✗

Type System

"The type of the compound does not determine the type of the components" [Jay09]

Example

$$t = \mathtt{c}\, x \rightarrow_{\{x:\mathsf{Bool}\}} \mathtt{if}\ x\ \mathtt{then}\ 1\ \mathtt{else}\ 0$$
$$\mid\ \mathtt{c}\, y \rightarrow_{\{y:\mathsf{Nat}\}}\ y + 1$$

- Singleton types ($\mathbb{c}$)
- Type application (@) [Pet11]
- Union types ($\oplus$)
- Recursive types (μ)

$$\begin{array}{rcl} \mathsf{Bool} & \triangleq & \mathbb{true} \oplus \mathbb{false} \\ \mathsf{Nat} & \triangleq & \mu X.\mathbb{zero} \oplus \mathbb{suc}\ @\ X \\ t & : & (\mathbb{c}\ @\ \mathsf{Bool} \oplus \mathbb{c}\ @\ \mathsf{Nat}) \supset \mathsf{Nat} \end{array}$$

Compatibility

We take advantage of the evaluation order...

Given $(p \rightarrow_\theta s \mid q \rightarrow_{\theta'} t)$

- If $p \lhd q$[1] then $q \rightarrow_{\theta'} t$ is a dead branch

[1] $p \lhd q$ iff $\exists \sigma . \sigma p = q$

Compatibility

We take advantage of the evaluation order...

Given $(p \to_\theta s \mid q \to_{\theta'} t)$

- If $p \lhd q$[1] then $q \to_{\theta'} t$ is a dead branch
- If not

	$p\vert_\pi$	$q\vert_\pi$	
a.		y	restriction required
b.	c	d	no overlapping ($q \not\lhd p$)
c.		$q_1\ q_2$	no overlapping
d.	$p_1\ p_2$	y	restriction required
e.		d	no overlapping

[1] $p \lhd q$ iff $\exists \sigma . \sigma p = q$

Compatibility

We take advantage of the evaluation order...

Given $(p \to_\theta s \mid q \to_{\theta'} t)$

- If $p \lhd q$[1] then $q \to_{\theta'} t$ is a dead branch
- If not

	$p\vert_\pi$	$q\vert_\pi$	
a.		y	restriction required
b.	c	d	no overlapping ($q \not\lhd p$)
c.		$q_1\ q_2$	no overlapping
d.	$p_1\ p_2$	y	restriction required
e.		d	no overlapping

...and state this in the *compatibility predicate*, $\mathrm{ovl}(p : A, q : B)$.

[1] $p \lhd q$ iff $\exists \sigma . \sigma p = q$

Compatibility

Example (Case a.)

$$f \rightarrow_{\{f:\mathsf{Bool}\supset A\}} \left(\begin{array}{lll} \mathtt{vl}\, z & \rightarrow_{\{z:\mathsf{Bool}\}} & \mathtt{vl}\,(f\, z) \\ \mid\ x\, y & \rightarrow_{\{x:\mathbb{v},y:\mathsf{Nat}\}} & x\, y \end{array} \right)$$

	$p\vert_\pi$	$q\vert_\pi$	
a.		y	restriction required
b.	c	d	no overlapping ($q \not\trianglelefteq p$)
c.		$q_1\ q_2$	no overlapping
d.	$p_1\ p_2$	y	restriction required
e.		d	no overlapping

Compatibility

Example (Case d.)

$$f \rightarrow_{\{f:\mathbb{v}\mathbb{l}\supset \mathsf{Bool}\supset A\}} g \rightarrow_{\{g:\mathbb{v}\mathbb{l}@\mathsf{Nat}\supset A\}} \left(\begin{array}{ll} x\,y & \rightarrow_{\{x:\mathbb{v}\mathbb{l},y:\mathsf{Bool}\}} f\,x\,y \\ \mid z & \rightarrow_{\{z:\mathbb{v}\mathbb{l}@\mathsf{Nat}\}} g\,z \end{array} \right)$$

	$p\vert_\pi$	$q\vert_\pi$	
a.		y	restriction required
b.	c	d	no overlapping ($q \not\trianglelefteq p$)
c.		$q_1\ q_2$	no overlapping
d.	$p_1\ p_2$	y	restriction required
e.		d	no overlapping

Compatibility

Definition (Compatibility)

$p : A$ and $q : B$ are *compatible* iff

$$\mathrm{ovl}(p : A, q : B) \implies B \preceq_\mu A$$

Typing scheme

$$\frac{(\theta_i \vdash_{\mathrm{p}} p_i : A_i)_{i \in 1..n} \quad [p_i : A_i]_{i \in 1..n} \text{compatible} \quad (\Gamma, \theta_i \vdash s_i : B)_{i \in 1..n}}{\Gamma \vdash (p_i \to_{\theta_i} s_i)_{i \in 1..n} : \bigoplus_{i \in 1..n} A_i \supset B}$$

Compatibility

Lemma (Compatibility Lemma)

Suppose $\Gamma \rhd u : B$, $\theta \rhd_{\mathsf{p}} p : A$, $\theta' \rhd_{\mathsf{p}} q : B$ *and* $\{\!\{u/p\}\!\}$ *is successful. Then,* $\mathsf{ovl}(p : A, q : B)$ *holds.*

Given a well-typed application

$$(p \to_{\theta} s \mid q \to_{\theta'} t)\, u$$

the lemma states that a branch should be more general than those to its left, or a dead branch otherwise ($B \preceq_{\mu} A$ by compatibility).

Safety

Reduction preserves typing

Proposition (Subject Reduction)

If $\Gamma \rhd s : A$ *and* $s \to s'$*, then* $\Gamma \rhd s' : A$*.*

And it does not get stuck

Proposition (Progress)

If $\rhd s : A$ *and* s *is not a value, then* $\exists s'$ *s.t.* $s \to s'$*.*

Conclusions

- We proposed a type system for a calculus that supports *path polymorphism*
- The two fundamental properties were proved to hold: *subject reduction* and *progress*
- We developed a notion of pattern *compatibility* that turns out to be crucial for the achieved results
- Invertibility of subtyping of recursive types in the presence of associative, commutative and idempotent unions was addressed too

Other work

Type-checking[2]

- Syntax-directed formalization of the type system
- Implementation of type-checking algorithms
  - Disambiguation of typing rules
  - Equivalence and subtype checking
  - Compatibility check
- Efficiency of typing
  - Checking equivalence and subtyping modulo ACI operators [JPZ02][DPR05]

[2] Juan Edi's MSc Thesis at UBA

Work in progress

Strong Normalization

- Recursive types are not SN

$$\vdash Y_C : (A \supset A) \supset A$$

- All known restrictions work with *weak equivalence*
  - Our invertibility result holds for *strong equivalence*
- SN with strong equivalence has been an open problem for over 15 years [Cop98]
- Options
  - Analyze SN with strong equivalence for recursive types in a simplified framework
  - Revisit our results considering weak type equivalence

Future Work

- Polymorphic type extensions
  - Intersection types
  - Parametric polymorphism *à la* System $F_{<:}$
- Curry–Howard correspondence for the proposed system
- Comparison with Dagand's work in *Ornaments*
- Comparison with Datatype-Generic Programming based on Category Theory
- Extend this analysis to the dynamic patterns case

References I

Mario Coppo.
Recursive types: the syntactic and semantic approaches.
In *Theories of Types and Proofs*, pages 16–41. Kyoto University, 1998.

Roberto Di Cosmo, François Pottier, and Didier Rémy.
Subtyping recursive types modulo associative commutative products.
In *Seventh International Conference on Typed Lambda Calculi and Applications (TLCA'05)*, Nara, Japan, April 2005.

C. Barry Jay.
Pattern Calculus: Computing with Functions and Structures.
Springer, 2009.

C. Barry Jay and Delia Kesner.
First-class patterns.
J. Funct. Program., 19(2):191–225, 2009.

References II

Somesh Jha, Jens Palsberg, and Tian Zhao.
Efficient type matching.
In Mogens Nielsen and Uffe Engberg, editors, *Foundations of Software Science and Computation Structures, 5th International Conference, FOSSACS 2002 (ETAPS 2002) Grenoble, France, April 8-12, 2002, Proceedings*, volume 2303 of *LNCS*, pages 187–204. Springer, 2002.

Barbara Petit.
Semantics of typed lambda-calculus with constructors.
Logical Methods in Computer Science, 7(1), 2011.

CAP

Patterns, terms, data structures and matchable forms

$$
\begin{array}{lcll}
p & ::= & x & \text{(matchable)} \\
 & | & \mathsf{c} & \text{(constant)} \\
 & | & p\,p & \text{(compound)}
\end{array}
\qquad
\begin{array}{lcll}
t & ::= & x & \text{(variable)} \\
 & | & \mathsf{c} & \text{(constant)} \\
 & | & t\,t & \text{(application)} \\
 & | & p \to_\theta t \mid \ldots \mid p \to_\theta t & \text{(abstraction)}
\end{array}
$$

$$
\begin{array}{lcll}
d & ::= & \mathsf{c} & \text{(constant)} \\
 & | & d\,t & \text{(compound)}
\end{array}
\qquad
\begin{array}{lcll}
m & ::= & d & \text{(data struct)} \\
 & | & p \to_\theta t \mid \ldots \mid p \to_\theta t & \text{(abstraction)}
\end{array}
$$

CAP

Matching operation

$$
\begin{array}{lcll}
\{\!\{u/x\}\!\} & \triangleq & \{u/x\} & \\
\{\!\{\mathtt{c}/\mathtt{c}\}\!\} & \triangleq & \{\} & \\
\{\!\{u\,v/p\,q\}\!\} & \triangleq & \{\!\{u/p\}\!\} \uplus \{\!\{v/q\}\!\} & \text{if } u\,v \text{ is a } \textit{matchable form} \\
\{\!\{u/p\}\!\} & \triangleq & \mathtt{fail} & \text{if } u \text{ is a } \textit{matchable form} \\
\{\!\{u/p\}\!\} & \triangleq & \mathtt{wait} &
\end{array}
$$

Reduction rule

$$
\frac{\{\!\{u/p_i\}\!\} = \mathtt{fail} \text{ for all } i < j \quad \{\!\{u/p_j\}\!\} = \sigma_j \quad j \in 1..n}{(p_i \to_{\theta_i} s_i)_{i \in 1..n}\, u \to \sigma_j s_j}\ (\beta)
$$

Types

Datatypes and types syntax

$$
\begin{array}{rcll}
D & ::= & \alpha & \text{(datatype variable)} \\
 & | & \mathbb{c} & \text{(atom)} \\
 & | & D \mathbin{@} A & \text{(compound)} \\
 & | & D \oplus D & \text{(union)} \\
 & | & \mu\alpha.D & \text{(recursion)}
\end{array}
$$

$$
\begin{array}{rcll}
A & ::= & X & \text{(type variable)} \\
 & | & D & \text{(datatype)} \\
 & | & A \supset A & \text{(type abstraction)} \\
 & | & A \oplus A & \text{(union)} \\
 & | & \mu X.A & \text{(recursion)}
\end{array}
$$

Type equivalence[3]

$$\frac{}{\vdash A \oplus A \simeq_\mu A}\ \text{(e-idem)} \qquad \frac{}{\vdash A \oplus B \simeq_\mu B \oplus A}\ \text{(e-comm)}$$

$$\frac{}{\vdash A \oplus (B \oplus C) \simeq_\mu (A \oplus B) \oplus C}\ \text{(e-assoc)}$$

$$\frac{}{\vdash \mu V.A \simeq_\mu \{\mu V.A/V\}\, A}\ \text{(e-fold)}$$

$$\frac{\vdash A \simeq_\mu \{A/V\}\, B \quad \mu V.B \text{ contractive}}{\vdash A \simeq_\mu \mu V.B}\ \text{(e-contr)}$$

[3] Reflexivity, transitivity, symmetry and congruence rules omitted.

Subtyping rules[4]

A **subtyping context** consists of assumptions of the form $X \preceq Y$

$$\frac{}{\Sigma, V \preceq_\mu W \vdash V \preceq_\mu W}\ \text{(s-hyp)} \qquad \frac{\vdash A \simeq_\mu B}{\Sigma \vdash A \preceq_\mu B}\ \text{(s-eq)}$$

$$\frac{\Sigma \vdash A \preceq_\mu C \quad \Sigma \vdash B \preceq_\mu C}{\Sigma \vdash A \oplus B \preceq_\mu C}\ \text{(s-union-l)}$$

$$\frac{\Sigma \vdash A \preceq_\mu B}{\Sigma \vdash A \preceq_\mu B \oplus C}\ \text{(s-union-r1)} \qquad \frac{\Sigma \vdash A \preceq_\mu C}{\Sigma \vdash A \preceq_\mu B \oplus C}\ \text{(s-union-r2)}$$

$$\frac{\Sigma, V \preceq_\mu W \vdash A \preceq_\mu B \quad W \notin \mathrm{fv}(A) \quad V \notin \mathrm{fv}(B)}{\Sigma \vdash \mu V.A \preceq_\mu \mu W.B}\ \text{(s-rec)}$$

[4] Reflexivity, transitivity and congruence rules omitted.

Compatibility

$$
\begin{array}{rcll}
a\|_\epsilon & \triangleq & \{a\} & \\
(A_1 \star A_2)\|_\epsilon & \triangleq & \{\star\}, & \star \in \{\supset, @\} \\
(A_1 \star A_2)\|_{i\pi} & \triangleq & A_i\|_\pi, & \star \in \{\supset, @\}, i \in \{1, 2\} \\
(A_1 \oplus A_2)\|_\pi & \triangleq & A_1\|_\pi \cup A_2\|_\pi & \\
(\mu V.A')\|_\pi & \triangleq & (\{\mu V.A'/V\}\, A')\|_\pi &
\end{array}
$$

Maximal positions

$$\mathsf{maxpos}(P) \triangleq \left\{\pi \in P \mid \nexists \pi' \in P.\pi' = \pi\pi'' \wedge \pi'' \neq \epsilon\right\}$$

Mismatching positions

$$\mathsf{mmpos}(p, q) \triangleq \{\pi \mid \pi \in \mathsf{maxpos}(\mathsf{pos}(p) \cap \mathsf{pos}(q)) \wedge p|_\pi \not\trianglelefteq q|_\pi\}$$

Compatibility

$p : A$ is *compatible* with $q : B$ ($p : A \lll q : B$) iff:

- $p \lhd q \implies B \preceq_\mu A$
- $p \not\lhd q \implies (\forall \pi \in \mathrm{mmpos}(p, q)\, .A\|_\pi \cap B\|_\pi \neq \varnothing) \implies B \preceq A.$

Alternately, let

$$\mathrm{ovl}(p : A, q : B) \triangleq p \lhd q \vee (\forall \pi \in \mathrm{mmpos}(p, q)\, .A\|_\pi \cap B\|_\pi \neq \varnothing)$$

Then,

$$p_i : A_i \lll p_j : A_j \quad \text{iff} \quad \mathrm{ovl}(p : A, q : B) \implies B \preceq_\mu A$$

A list of patterns $[p_i : A_i]_{i \in 1..n}$ is compatible if

$$\forall i, j \in 1..n. i < j \implies p_i : A_i \lll p_j : A_j$$

Typing schemes

Patterns are typed separately with special judgment $\vdash_{\mathsf{p}}$

$$\frac{\theta(x) = A}{\theta \vdash_{\mathsf{p}} x : A}\ \text{(p-match)}$$

$$\frac{}{\theta \vdash_{\mathsf{p}} \mathsf{c} : \mathbb{c}}\ \text{(p-const)}$$

$$\frac{\theta \vdash_{\mathsf{p}} p : D \quad \theta \vdash_{\mathsf{p}} q : A}{\theta \vdash_{\mathsf{p}} p\, q : D \mathbin{@} A}\ \text{(p-comp)}$$

Typing schemes

Rules for variables, data structures and subtyping

$$\frac{\Gamma(x) = A}{\Gamma \vdash x : A}\ \text{(t-var)}$$

$$\frac{}{\Gamma \vdash \mathsf{c} : \mathbb{c}}\ \text{(t-const)} \qquad \frac{\Gamma \vdash r : D \quad \Gamma \vdash u : A}{\Gamma \vdash r\, u : D \mathbin{@} A}\ \text{(t-comp)}$$

$$\frac{\Gamma \vdash s : A \quad \vdash A \preceq_{\mu} A'}{\Gamma \vdash s : A'}\ \text{(t-subs)}$$

The subtyping relation handles *union* and *recursive types*

Typing schemes

Special rules for abstraction and application

$$\frac{(\theta_i \vdash_{\mathsf{p}} p_i : A_i)_{i \in 1..n} \quad [p_i : A_i]_{i \in 1..n} \text{compatible} \quad (\Gamma, \theta_i \vdash s_i : B)_{i \in 1..n}}{\Gamma \vdash (p_i \to_{\theta_i} s_i)_{i \in 1..n} : \bigoplus_{i \in 1..n} A_i \supset B} \text{(t-abs)}$$

$$\frac{\Gamma \vdash r : \bigoplus_{i \in 1..n} A_i \supset B \quad \Gamma \vdash u : A_k \quad k \in 1..n}{\Gamma \vdash r\, u : B} \text{(t-app)}$$